

Génération de code intermédiaire

Cyril Rabat

`cyril.rabat@univ-reims.fr`

Licence 3 Informatique - Info0602 - Langages et compilation

2022-2023



Cours n°7

Génération de quadruplets

Étude des expressions booléennes, des tableaux et des conversions de type

Version 17 mars 2023

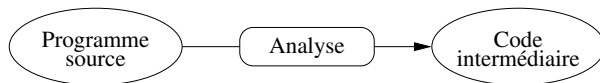
Table des matières

7 Génération de code intermédiaire

- Introduction
- Génération de quadruplets
- Expressions booléennes
- Tableaux
- Conversion de types

Génération de code intermédiaire

- Code assembleur dépendant de la machine
 - Copie des données (variables) dans les registres du processeur
 - Instructions suivant le jeu d'instructions du processeur
- Code intermédiaire = code assembleur indépendant de la machine
↪ «Assembleur universel»



Exemple d'un programme en C

Code en C

```
void main() {  
    int i, j;  
  
    j = 0;  
    i = 15;  
    j += i;  
}
```

Compilation

```
gcc -o exemple exemple.c
```

Désassembleur en ligne :

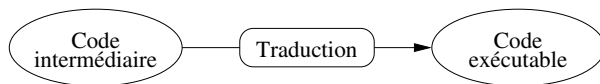
<https://onlinedisassembler.com/odaweb/>

Extrait du code assembleur

```
unknown main (unknown)  
  
push %rbp  
mov %rsp,%rbp  
movl $0x0,-0x8(%rbp)  
movl $0xf,-0x4(%rbp)  
mov -0x4(%rbp),%eax  
add %eax,-0x8(%rbp)  
pop %rbp  
retq  
nopw 0x0(%rax,%rax,1)
```

Pourquoi le code intermédiaire ?

- Possible de prévoir un optimiseur de code
↳ Indépendant de la machine
- Les arbres abstraits sont déjà des représentations intermédiaires
- L'objectif est de passer de la représentation arborescente à une représentation linéaire



Un code à 3 adresses

Définition : code à 3 adresses

*Un code à 3 adresses est une suite d'opérations de la forme :
 $x := y \text{ op } z$, avec op un opérateur.*

- 3 opérations de base :
 - ① Affectation
 - ② Branchement
 - ③ Appel de procédure

Opérations de base

- **Affectation** : 3 types

- $x := y \text{ op } z$: op un opérateur binaire (+, -, *, /, ...)
- $x := \text{op } y$: op un opérateur unaire (moins unaire, négation logique...)
- $x := y$: l'affectation

- **Branchement** : 2 types

- Inconditionnel : `aller_à L`, où L est une étiquette
- Conditionnel : `si x oprel y aller_à L`
 $\hookrightarrow$ oprel un opérateur relationnel (<, >, ≤, ≥, =, ≠)

- **Appel de procédure**

`param x1`

`param x2`

`...`

`param xn`

`appeler p, n`

$\hookrightarrow$ n est le nombre de paramètres effectifs

Adressages

- **Adressage indicé :**

- $A[i] := B$
- $B[i] := A[i]$

- **Adressage indirect :**

- $A := \&B$
- $A := *B$
- $*A := B$

Représentation du code à 3 adresses

Définition : quadruplet

Un quadruplet est une structure composée de 4 champs appelés op , $arg1$, $arg2$ et $résultat$.

Code à 3 adresses	Quadruplet
$x := y + z$	$+, y, z, x$
$x := -y$	$moinsu, y, , x$
$x := y$	$:=, y, , x$
$aller_à\ 10$	$aller_à, , , 10$
$si\ a < b\ aller_à\ 10$	$<, a, b, 10$

Exemple de génération d'un code à 3 adresses

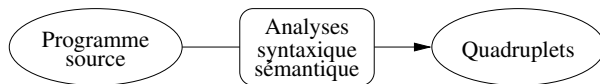
Soit l'instruction : $a := x * -y + c$

Résultat :

	op	arg1	arg2	résultat
(0)	moinsu	y		t1
(1)	*	x	t1	t2
(2)	+	t2	c	t3
(3)	:=	t3		a

Remarque : t1, t2 et t3 seront stockés dans la table des symboles

Génération de quadruplets dirigée par la syntaxe



- Les quadruplets sont une écriture en ligne de l'arbre syntaxique
- Pour chaque nœud interne de l'arbre, une variable temporaire doit être créée

Génération de code : expressions arithmétiques (1/4)

$$\begin{aligned} A &\rightarrow id := E \\ E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow id \mid (E) \mid cst \end{aligned}$$

Sémantique de $id := E$:

- 1 Évaluation (calcul) de la valeur de E avec l'état des variables avant affectation
- 2 Valeur obtenue placée dans la variable id

La valeur de E doit être stockée : un emplacement mémoire est nécessaire.

Génération de code : expressions arithmétiques (2/4)

Pour les symboles E, T et F, ajout d'un attribut `place` : `E.place`, `T.place` et `F.place`
↪ Emplacement mémoire qui contiendra la valeur de E, T et F

On suppose les 4 fonctions suivantes :

- `nouvTemp()` : renvoie un emplacement mémoire libre
- `place(id.lex, TS)` : renvoie l'entrée correspondant à l'identificateur `id.lex` dans la table des symboles (TS)
- `ajoute(id.lex, T.type, TS)` : alloue un emplacement mémoire à `id.lex` et crée l'entrée correspondante dans TS
- `gen(op, a, b, r)` : ajoute le quadruplet `(op, a, b, r)` à une liste (initialement vide)

Génération de code : expressions arithmétiques (3/4)

- Pour le moment, on ne tient pas compte du type et de l'ajout dans la table

```
A → id := E
{ gen(:=, E.place, , place(id.lex, TS) }
```

```
E → E1 + T
{
  E.place ← nouvTemp()
  gen(+, E1.place, T.place, E.place)
}
```

```
E → T
{ E.place ← T.place }
```

```
T → T1 * F
{
  T.place ← nouvTemp()
  gen(*, T1.place, F.place, T.place)
}
```

Génération de code : expressions arithmétiques (4/4)

```
T → F
{ T.place ← F.place }

F → (E)
{ F.place ← E.place }

F → id
{ F.place ← place(id.lex, TS) }

F → cst
{
  F.place ← nouvTemp()
  gen(cst, cst.lex, , F.place)
}
```

Génération de code : expressions booléennes

Il existe 2 méthodes d'évaluation des expressions booléennes :

- 1 Pascal
- 2 Fortran, C

Avec $exp1 \text{ op } exp2$:

- 1 Évaluation de $exp1$ et de $exp2$, puis évaluation du résultat
 - ↪ Génération de code plus facile
 - ↪ Identique aux expressions arithmétiques
- 2 $exp2$ est évalué si nécessaire
 - ↪ Si $op = \wedge$ et que $exp1$ est faux, $exp2$ n'est pas évalué

Première méthode

Pour analyser `exp1 op exp2` :

- Évaluation de `exp1`
- Résultat placé dans `exp1.place`
- Évaluation de `exp2`
- Résultat placé dans `exp2.place`
- Au moment de la réduction :
`(op, exp1.place, exp2.place, resultat.place)`

Deuxième méthode : exemple (1/4)

$$B \rightarrow B \text{ or } B_1 \mid B_1$$

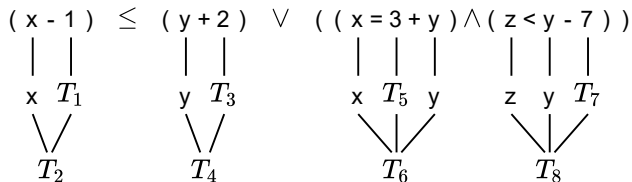
$$B_1 \rightarrow B_1 \text{ and } B_2 \mid B_2$$

$$B_2 \rightarrow \text{not } B_2 \mid (B) \mid \text{id} \mid (E \text{ OC } E)$$

$$\text{OC} \rightarrow < \mid \leq \mid > \mid \geq \mid = \mid \neq$$

$$E \rightarrow \dots$$

Exemple : $(x - 1) \leq (y * 2) \vee ((x = 3 + y) \wedge (z < y - 7))$



Deuxième méthode : exemple (2/4)

Code pour l'instruction suivante :

```
if((x-1) ≤ (y * 2) ∨ ((x = 3 + y) ∧ (z < y - 7)))
    x := y * 2 - z
else
    x := 7
```

Le test $(x - 1) \leq (y * 2)$:

(1)	cst	1		T_1	
(2)	-	x	T_1	T_2	
(3)	cst	2		T_3	
(4)	*	y	T_3	T_4	
(5)	≤	T_2	T_4	?	<i>Il faut exécuter le alors</i>
(6)	goto			7	<i>Il faut continuer l'évaluation</i>

Deuxième méthode : exemple (3/4)

Le test $x = 3 + y$:

(7)	cst	3		T_5	
(8)	+	T_5	y	T_6	
(9)	=	x	T_6	11	<i>Il faut exécuter $z < y - 7$</i>
(10)	goto			?	<i>Il faut exécuter le sinon</i>

Le test $z < y - 7$:

(11)	cst	7		T_7	
(12)	-	y	T_7	T_8	
(13)	<	z	T_8	?	<i>Il faut exécuter le alors</i>
(14)	goto			?	<i>Il faut exécuter le sinon</i>

Deuxième méthode : exemple (4/4)

Le alors : $x := y * 2 - z$

(15) `cst` 2 T_9

(16) `*` y T_9 T_{10}

(17) `-` T_{10} z T_{11}

(18) `:=` T_{11} x

(19) `goto` ? *Après le sinon*

Le sinon : $x := 7$

(20) `cst` 7 T_{12}

(21) `:=` T_{12} x

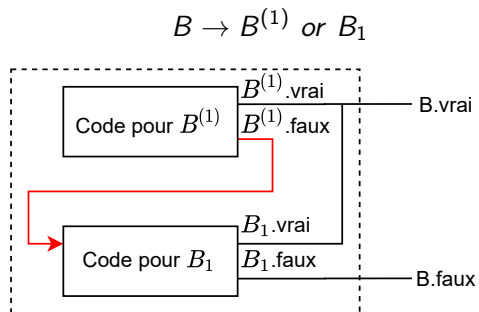
Il reste à compléter : le alors (15) et le sinon (20).

Deuxième méthode : les attributs

- Attributs synthétisés nécessaires : `B.vrai` et `B.faux`
 $\hookrightarrow$ Idem pour `B1` et `B2`
- Correspond à des listes de numéros de quadruplets dont les sauts sont à compléter
- `retour_arrière(l, n)` où `l` est une liste de quadruplets qu'il faut compléter par l'adresse de saut `n`
- `prochain_quad()` renvoie le numéro du prochain quadruplet qui sera généré
- `créer_liste(i)` crée une nouvelle liste contenant uniquement `i`

Il reste à définir les actions sémantiques

Deuxième méthode : décoration de la grammaire (1/8)



L'adresse du quadruplet inconnu doit être conservé dans un attribut

Deuxième méthode : décoration de la grammaire (2/8)

2 solutions :

$$\begin{aligned} \textcircled{1} \quad & B \rightarrow B \text{ OR } B_1 \\ & \text{OR} \rightarrow \text{or} \\ & \{ \text{OR.quad} \leftarrow \text{prochain_quad}() \} \end{aligned}$$

$$\begin{aligned} \textcircled{2} \quad & B \rightarrow B \text{ or } M B_1 \\ & M \rightarrow \epsilon \\ & \{ M.\text{quad} \leftarrow \text{prochain_quad}() \} \end{aligned}$$

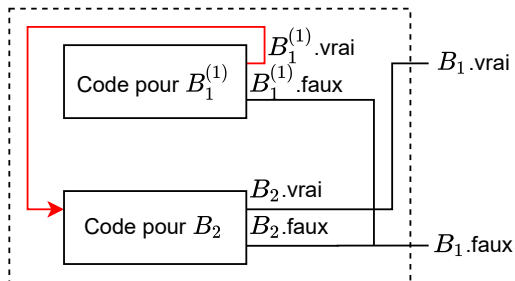
Dans la suite, nous choisissons la deuxième solution

Deuxième méthode : décoration de la grammaire (3/8)

```
B → B(1) or M B1
{
    retour_arrière(B(1).faux, M.quad)
    B.vrai ← fusion(B(1).vrai, B1.vrai)
    B.faux ← B1.faux
}

B → B1
{
    B.vrai ← B1.vrai
    B.faux ← B1.faux
}
```

Deuxième méthode : décoration de la grammaire (4/8)



```

 $B_1 \rightarrow B_1^{(1)} \text{ and } M \ B_2$ 
{
    retour_arrière( $B_1^{(1)}$ .vrai, M.quad)
     $B_1$ .vrai  $\leftarrow$   $B_2$ .vrai
     $B_1$ .faux  $\leftarrow$  fusion( $B_1^{(1)}$ .faux,  $B_2$ .faux)
}

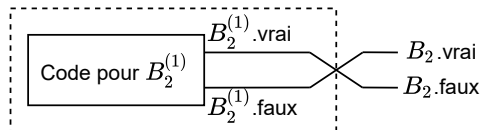
```

Deuxième méthode : décoration de la grammaire (5/8)

```
B1 → B2
{
  B1.vrai ← B2.vrai
  B1.faux ← B2.faux
}
```

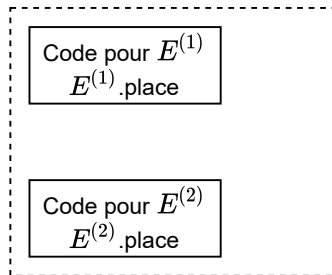
```
B2 → (B)
{
  B2.vrai ← B.vrai
  B2.faux ← B.faux
}
```

Deuxième méthode : décoration de la grammaire (6/8)



```
B2 → not B2(1)
{
  B2.vrai ← B2(1).faux
  B2.faux ← B2(1).vrai
}
```

Deuxième méthode : décoration de la grammaire (7/8)



```
B2 → E(1) OC E(2)
{
  B2.vrai ← créer_liste(prochain_quad())
  gen(OC, E(1).place, E(2).place, ?)
  B2.faux ← créer_liste(prochain_quad())
  gen(aller_à, , , ?)
}
```

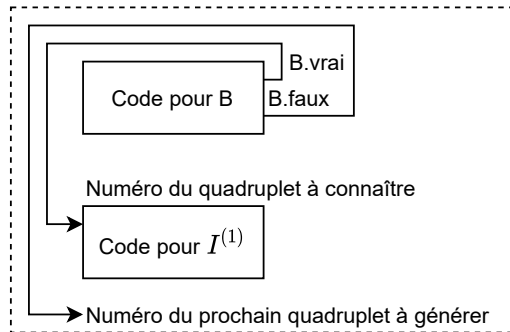
Deuxième méthode : décoration de la grammaire (8/8)

```
B2 → id
{
  Si type(id.lex, TS) = booléen Alors
    B2.vrai ← créer_liste(prochain_quad())
    gen(:=, vrai, place(id.lex, TS), ?)
    B2.faux ← créer_liste(prochain_quad())
    gen(aller_à, , ?)
  Sinon
    /* Erreur */
}
```

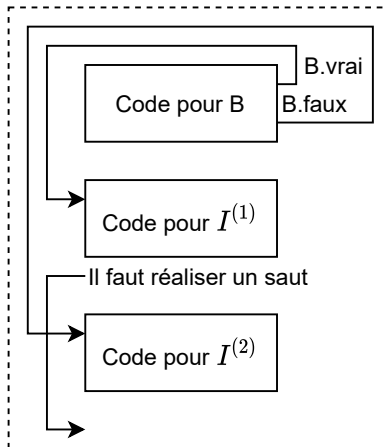
Les conditionnelles (1/3)

Soit la grammaire suivante :

$I \rightarrow \text{Si } B \text{ alors } I^{(1)}$
 $| \text{Si } B \text{ alors } I^{(1)} \text{ Sinon } I^{(2)}$



Les conditionnelles (2/3)



Les conditionnelles (3/3)

$I \rightarrow \text{Si } B \text{ alors } M \text{ I}$

```
{  
  retour_arrière(B.vrai, M.quad)  
  retour_arrière(B.faux, prochain_quad())  
}
```

$I \rightarrow \text{Si } B \text{ alors } M \text{ I Sinon } N \text{ I}$

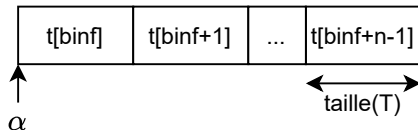
```
{  
  retour_arrière(B.vrai, M.quad)  
  retour_arrière(B.faux, N.quad)  
  retour_arrière(créer_liste(N.quad-1), prochain_quad())  
}
```

$N \rightarrow \epsilon$

```
{  
  gen(aller_à, , , ?)  
  N.quad  $\leftarrow$  prochain_quad()  
}
```

Les tableaux

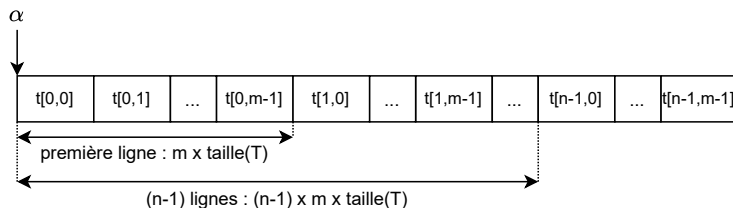
- Habituellement, les valeurs des tableaux sont stockées dans un bloc d'emplacements consécutifs
- Pour un tableau de type T de n éléments, on associe un espace mémoire suffisant pour contenir n valeurs de type T
- Lors de la déclaration, on stocke dans la TS le type des cases du tableau T , l'adresse de début du tableau (α) et (éventuellement) le nombre d'éléments
- Pour accéder à un élément $t[i]$: $\alpha + (i - \text{binf}) \times \text{taille}(T)$



Les tableaux à 2 dimensions

- Deux solutions :
 - ① Stockage colonne par colonne (Fortran)
 - ② Stockage ligne par ligne (C)

Exemple avec $t[n, m]$, $binf_1 = 0$ et $binf_2 = 0$:



$t[i,j]$ est stocké en position $\alpha + (i \times m + j) \times \text{taille}(T)$

Cas général (1/3)

Soit le tableau $t[n_1, n_2, \dots, n_k]$ avec $\text{binf}_i = 0, \forall i \in [1, k]$

$t[0, i_2, \dots, i_k]$	$t[1, i_2, \dots, i_k]$	...	$t[n_k-1, i_2, \dots, i_k]$
-------------------------	-------------------------	-----	-----------------------------

Quelle est la taille d'un bloc ?



Cas général (2/3)

Soit le tableau $t[n_1, n_2, \dots, n_k]$ avec $\text{binf}_i = 0, \forall i \in [1, k]$

$t[0, i_2, \dots, i_k]$	$t[1, i_2, \dots, i_k]$	...	$t[n_k-1, i_2, \dots, i_k]$
-------------------------	-------------------------	-----	-----------------------------

La taille d'un bloc : $n_2 \times n_3 \times \dots \times n_k \times \text{taille}(T)$



Décalage à réaliser :

$$\begin{aligned}
 \text{décalage}(t[i_1, \dots, i_k]) = & \alpha + \\
 & i_1 \times n_2 \times \dots \times n_k \times \text{taille}(T) + \\
 & i_2 \times n_3 \times \dots \times n_k \times \text{taille}(T) + \\
 & \dots + \\
 & i_{k-1} \times n_k \times \text{taille}(T) + \\
 & i_k \times \text{taille}(T)
 \end{aligned}$$

Cas général (3/3)

Le décalage peut être calculé à l'aide du schéma de Horner :

$$(((\dots(((i_1 - \text{binf}_1) \times n_2 + (i_2 - \text{binf}_2)) \times n_3 + (i_3 - \text{binf}_3))\dots) \times n_k + (i_k - \text{binf}_k)) \times \text{taille}(T) + \alpha$$

Que l'on peut écrire sous la forme suivante :

$$(((\dots((i_1 \times n_2 + i_2) \times n_3 + i_3)\dots) \times n_k + i_k) \times \text{taille}(T) + \alpha - (\dots((\text{binf}_1 \times n_2 + \text{binf}_2) \times n_3 + \text{binf}_3)\dots) \times n_k + \text{binf}_k) \times \text{taille}(T)$$

Le décalage peut être calculé partiellement à la compilation :

↪ La deuxième ligne est une constante

↪ Avec $\text{binf}_i = 0, \forall i \in [0, k]$, la constante est α

La génération de code (1/7)

- Problème : relier le calcul du décalage
- Par exemple avec la grammaire suivante :
 $A \rightarrow id \mid id[LE]$
 $LE \rightarrow LE, E \mid E$
 $\hookrightarrow$ Les dimensions du tableau ne sont pas accessibles au fur-et-à-mesure du groupement de LE

Solution

- La grammaire transformée :
 $A \rightarrow id \mid LE$
 $LE \rightarrow LE, E \mid id[E]$
- Pour LE, nous avons besoin des attributs :
 - *tableau* : le nom du tableau ou son entrée dans TS
 - *n* : le nombre d'indices dans LE
 - *decal* : la valeur du décalage calculé

La génération de code (2/7)

Avec l'affectation, nous obtenons la grammaire suivante :

$$\begin{aligned} A &\rightarrow \text{id} := E \mid \text{LE}] := E \\ \text{LE} &\rightarrow \text{LE}, E \mid \text{id}[E \\ E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{cst} \mid \text{id} \mid \text{LE}] \end{aligned}$$

La génération de code (3/7)

```
LE → id [ E
{
  LE.tableau ← id.lex
  LE.n ← 1
  LE.decal ← E.place
}
```

- Le nom du tableau est associé à l'expression indiquée la plus à gauche
- L'attribut *tableau* est alors associé à LE
- n représente le nombre d'expressions donc le nombre de dimensions du tableau
- Pour la suite, utilisation de la fonction `borne(tab, j, TS)`
↪ Calcule le nombre d'éléments n_j du tableau *tab*

La génération de code (4/7)

- EL produit les m premiers indices de $T[i_1, i_2, \dots, i_k]$
- Le code engendré permet de calculer :

$$(..((i_1 \times n_2 + i_2) \times n_3 + i_3) \dots) \times n_m + i_m$$

- Par récurrence :

$$e_1 = i_1$$

$$e_m = e_{m-1} \times n_m + i_m$$

La génération de code (5/7)

```
LE  $\rightarrow$  LE(1), E
{
  LE.tableau  $\leftarrow$  LE(1).tableau
  LE.n  $\leftarrow$  LE(1).n + 1
  T  $\leftarrow$  nouvTemp()
  gen(cst, borne(LE.tableau, LE.n, TS), , T)
  gen(*, LE(1).decal, T, LE(1).decal)
  gen(+, LE(1).decal, E.place, LE(1).decal)
  LE.decal  $\leftarrow$  LE(1).decal
}
```

La génération de code (6/7)

- La fonction $c(tab, TS)$ retourne la valeur de la constante correspondant au tableau tab
- La fonction $taille(tab, TS)$ retourne $taille(T)$

```
F → LE]
{
  u ← nouvTemp()
  gen(cst, c(LE.tableau, TS), , u)
  v ← nouvTemp()
  gen(cst, taille(LE.tableau, TS), , v)
  gen(*, LE.decal, v, v)
  F.place ← nouvTemp()
  gen(:=[], u, v, F.place)
}
```

La génération de code (7/7)

```
A → LE] := E
{
  u ← nouvTemp()
  gen(cst, c(LE.tableau, TS), , v)
  v ← nouvTemp()
  gen(cst, taille(LE.tableau, TS), , v)
  gen(*, LE.decal, v, v)
  gen([]:=, u, v, E.place)
}
```

L'attribut *decal* est nul pour un nom simple :

```
F ← id
{
  F.place ← place(id.lex, TS)
  F.decal = nul
}
```

Exemple

- Supposons un tableau de $n_1 = 10$ cases par $n_2 = 20$ avec $binf_1 = 1$ et $binf_2 = 1$, $taille(T) = 4$
- La constante retournée par la fonction c est $(1 \times 20 + 1) \times 4 = 84$
- Avec l'instruction $x := T[y, z]$, le code généré est le suivant :

```

(1)    *    y    20    t1
(2)    +    t1    z    t1
(3)    cst    k                t2 /* k correspond à α - 84 */
(4)    *    4    t1    t3
(5)    :=[]    t2    t3    t4
(6)    :=    t4                x

```

Conversions de type

- La prise en compte des types nécessite de modifier :
 - Les règles sémantiques (vérification des types)
 - La génération du code intermédiaire (conversion)
- Il faut le faire pour chaque opération
- Cela nécessite de définir les opérations "typées", les conversions de types
- Le nombre de types différents multiplie le nombre de cas à gérer !

Du point de vue sémantique

Avec deux types entier et réel :

```
E → E(1) + E(2)
{
  Si E(1).type = entier et E(2).type = entier Alors
    E.type ← entier
  Sinon
    E.type ← réel
}
```


La génération de code

- Ajout des opérations entier+ et réel+
- Opérations de conversion EntierVersRéal

```

E → E(1) + E(2)
{
  E.place ← nouvTemp()
  Si E(1).type = entier et E(2).type = entier Alors
    gen(entier+, E(1).place, E(2).place, E.place)
    E.type ← entier
  Sinon Si E(1).type = réel et E(2).type = réel Alors
    gen(réel+, E(1).place, E(2).place, E.place)
    E.type ← réel
  Sinon Si E(1).type = entier et E(2).type = réel Alors
    u ← nouvTemp()
    gen(EntierVersRéal, E(1).place, , u)
    gen(réel+, u, E(2).place, E.place)
    E.type ← réel
  Sinon Si E(1).type = réel et E(2).type = entier Alors
    u ← nouvTemp()
    gen(EntierVersRéal, E(2).place, , u)
    gen(réel+, E(1).place, u, E.place)
    E.type ← réel
}

```